□ LEOPARDO RH

DOSSIER 1 — ANALYSE COMPLÈTE & ÉLÉMENTS MANQUANTS

État au 29 Mars 2026 | Avant démarrage du code

CATÉGORIE	STATUT	PRIORITÉ
Conception (23 docs)	✅Complet à 92%	—
Wireframes (20 écrans HTML)	✅Complet	—
API Contrats	□ 42% (29/70 endpoints)	□ BLOQUANT Jules
Arborescence code	✅Définie	—
Code backend (Laravel)	❌0%	□ Démarrer maintenant
Code mobile (Flutter)	❌0%	□ En parallèle backend
Code frontend web (Vue.js)	❌0%	□ Phase 2 (sem. 15+)
Modèles Dart Flutter	❌Manquants	□ Nécessaire Jules sem. 1
Stratégie cache Redis	✅Documentée	—
CI/CD & Branches Git	✅Documenté	□ Fichiers yaml à créer
Stratégie de tests	□ Partielle	□ Pendant code
Multi-tenant hybride	✅Documenté	—
Landing page + Tunnel vente	□ Spécifié, non codé	□ Phase 2

1. CE QUI EST SOLIDE — VOTRE BASE EST EXCELLENTE

Avant de lister les manques, il est important de noter que votre niveau de conception est bien au-dessus de la moyenne pour un projet à ce stade. Voici ce qui est prêt et de qualité :

DOCUMENT / ÉLÉMENT	QUALITÉ
CDC v3.0 + DCT v1.0 (PDF)	✅Spécifications fonctionnelles et techniques complètes
ERD complet (03_ERD_COMPLET.md)	✅Relations tables bien modélisées
Schéma SQL complet (07_SCHEMA_SQL.sql)	✅39 183 lignes — très détaillé
RBAC 7 rôles (10_RBAC_COMPLET.md)	✅Matrice permissions par module
Multi-tenant hybride (11_MULTITENANCY.md)	✅Mode schéma + shared selon plan
Sécurité (12_SECURITY_SPEC.md)	✅Règles définies
Règles métier (09_REGLES_METIER.md)	✅Calculs paie, absences, HS documentés
Prompt maître Claude Code	✅Stack technique verrouillée + règles de code
Prompt maître Jules Flutter	✅Architecture Riverpod + GoRouter définie
Wireframes HTML complet (147 KB)	✅20+ écrans interactifs
Sprint 0 Checklist (04_SPRINT_0.md)	✅Setup o2switch, Firebase, stores définis
Guide ajout pays (15_GUIDE_AJOUT_PAYS.md)	✅Extension Afrique documentée
Templates notifications (14_NOTIF_TEMPLATES.md)	✅Push, email, SMS définis

2. MANQUE CRITIQUE 1 — API CONTRATS INCOMPLETS (BLOQUANT)

Statut actuel : 29 endpoints documentés sur ~70 nécessaires. Jules ne peut pas développer les écrans sans ces contrats.

Endpoints manquants par module

□ Auth — PRIORITÉ ABSOLUE (Semaine 1)

Endpoint	Méthode	Raison critique
GET /auth/me	GET	Profil utilisateur connecté — nécessaire au démarrage de tout écran Flutter
POST /auth/device/fcm	POST	Enregistrement token FCM à chaque login pour les push notifications
DELETE /auth/device/fcm	DELETE	Suppression token FCM à la déconnexion

□ Pointage — PRIORITÉ ABSOLUE (Semaine 1)

Endpoint	Méthode	Raison critique
GET /attendance/today	GET	⚡ ULTRA-CRITIQUE : Jules ne peut pas savoir si l'employé a déjà pointé. Retourne null ou le log du jour.
GET /attendance/{employee_id}	GET	Historique pointages d'un employé spécifique

□ Configuration entreprise — ENTIÈREMENT ABSENTE (Semaine 3)

Les endpoints CRUD pour départements, postes, plannings et sites sont complètement absents des contrats.

Ressource	Endpoints manquants	Impact
/departments	GET, POST, PUT/{id}, DELETE/{id}	Pas de config org sans ça
/positions	GET, POST, PUT/{id}, DELETE/{id}	Postes non gérables
/schedules	GET, POST, PUT/{id}, DELETE/{id}	Plannings non configurables
/sites	GET, POST, PUT/{id}, DELETE/{id}	GPS zones impossibles

□ Autres modules partiellement documentés (Semaines 5-8)

Module	Endpoints manquants

Employés	GET /{id}, PUT /{id}, DELETE /{id}, POST /import, GET /{id}/payslips
Absences	GET /absences, GET /{id}, PUT /{id}/cancel, CRUD /absence-types
Avances	GET /advances, GET /{id}, PUT /{id}/reject, PUT /{id}/repayment
Tâches	GET /tasks, GET /{id}, PUT, DELETE, GET /{id}/comments, CRUD /projects
Paie	GET /payroll, GET /{id}, GET /{id}/pdf, GET /status/{job_id}
Notifications	TOUT manque : GET, PUT /{id}/read, PUT /read-all, GET /count
Évaluations	TOUT manque : GET, POST, GET /{employee_id}, PUT /{id}/self
Rapports	GET /reports/absences, /payroll, /performance
Super Admin	CRUD plans, billing, stats plateforme, reactivate/extend
Paramètres	PUT /settings, PUT /apply-hr-model, GET /hr-models
Appareils ZKTeco	CRUD /devices, POST /devices/{id}/test-connection

3. MANQUE 2 — MODÈLES DART FLUTTER (Nécessaire Semaine 1-2)

Jules a besoin des classes Dart avec fromJson()/toJson() pour pouvoir développer sans API réelle (mock data). C'est ce qui permet le développement mobile en parallèle du backend.

Classe Dart	Champs clés	Priorité
Employee	id, matricule, firstName, lastName, role, leaveBalance, photoUrl	□ Semaine 1
AttendanceLog	id, date, checkIn (DateTime), checkOut, hoursWorked, status, method	□ Semaine 1
Absence	id, type, startDate, endDate, status, approvedBy	□ Semaine 2
SalaryAdvance	id, amount, reason, status, repaymentPlan	□ Semaine 3
Task	id, title, description, priority, status, assigneeId, dueDate	□ Semaine 4
PayrollSlip	id, month, year, grossSalary, netSalary, status, pdfUrl	□ Semaine 6
AppNotification	id, title, body, type, isRead, createdAt	□ Semaine 5
Project	id, name, description, status, managerId	□ Semaine 4
Evaluation	id, employeeId, period, scores, comments, selfEvalDone	□ Semaine 8

□ Règle critique : les dates/heures doivent TOUJOURS être des DateTime, jamais des String. Exemple : checkIn: json['check_in'] != null ? DateTime.parse(json['check_in']) : null

4. MANQUE 3 — POINTS TECHNIQUES À CLARIFIER AVANT LE CODE

4.1 Multi-tenant : Choix du mode par défaut

Votre PROMPTcONCEPTion.md indique que vous souhaitez pouvoir choisir 'expressément' le mode isolation physique (schéma) ou logique (shared). C'est bien documenté dans 11_MULTITENANCY_STRATEGY.md. Confirmation du fonctionnement :

Plan	Mode par défaut	Changeable ?
Trial / Starter	Shared (logique via company_id)	✅Upgrade vers Schema possible
Business / Enterprise	Schema isolé (physique)	✅Par défaut — sur demande

✅Ce point est résolu dans votre documentation. Le TenantMiddleware gère les deux modes via companies.tenancy_type.

4.2 Monorepo vs Multi-repos

Vous avez confirmé vouloir un seul repo (monorepo). C'est la bonne décision pour que les agents IA aient une vision globale. Structure confirmée :

- leopardo-rh/ → racine du monorepo
- leopardo-rh/api/ → Backend Laravel 11 + Vue.js (Inertia)
- leopardo-rh/mobile/ → Flutter Android + iOS
- leopardo-rh/docs/ → Toute la documentation de conception

4.3 Landing page et tunnel de vente

Spécifié dans 18_MARKETING_ET_VENTES.md. Décision validée : intégration dans le dossier api/ via Blade (SEO-friendly). À développer en Phase 2 (semaine 15+).

4.4 Mock Data Flutter — Comment Jules développe sans API

Solution à mettre en place dès la semaine 1 : créer des fichiers JSON de mock dans mobile/assets/mock/ correspondant exactement aux réponses API documentées. Jules configure Dio pour pointer sur ces fichiers en mode développement.

Fichier Mock	Simule
mock_auth_login.json	Réponse POST /auth/login avec token fictif
mock_attendance_today.json	GET /attendance/today — null ou log du jour
mock_attendance_history.json	Liste de 30 jours de pointages
mock_absences.json	Liste absences + solde congés
mock_tasks.json	Liste tâches assignées à l'employé

mock_payroll.json	Bulletins des 6 derniers mois
mock_notifications.json	5 notifications avec 2 non lues
mock_employee_profile.json	GET /auth/me — profil complet

5. MANQUE 4 — FICHIERS À CRÉER AVANT / PENDANT LE CODE

5.1 Fichiers de configuration manquants

Fichier	Où	Quand créer
.github/workflows/tests.yml	Racine monorepo	☐ Avant premier commit
.github/workflows/deploy.yml	Racine monorepo	☐ Semaine 8-10
api/CHANGELOG.md	Dossier api/	☐ Avant release 0.1.0
api/.env.example	Dossier api/	☐ Semaine 1
mobile/assets/mock/*.json	Dossier mobile/	☐ Semaine 1
api/storage/app/firebase-credentials.json	Dossier api/	☐ Sprint 0 (obtenu Firebase)
mobile/android/app/google-services.json	Dossier mobile/	☐ Sprint 0 (obtenu Firebase)

5.2 Complétions documentaires à faire

Document	Ce qui manque
02_API_CONTRATS.md	~41 endpoints non documentés (voir Section 2)
20_MODELES_DART.md	Classes Dart fromJson/toJson complètes (voir Section 3)
16_STRATEGIE_TESTS.md	Scénarios de test détaillés (actuellement très court)
README.md (monorepo)	Conventions Git + setup local développeur

6. ORDRE DE RÉSOLUTION RECOMMANDÉ

□ MAINTENANT — Avant la première ligne de code

- **Compléter API contrats : GET /auth/me, POST /auth/device/fcm, GET /attendance/today**
- Créer la stratégie de branches Git dans le README du monorepo
- Créer les fichiers mock JSON pour Jules (8 fichiers — voir tableau 5.1)
- Créer les modèles Dart Employee et AttendanceLog dans 20_MODELES_DART.md
- Créer .github/workflows/tests.yml (CI automatique)

□ Semaines 1-2 — Infrastructure

- Claude Code : Setup Laravel 11, PostgreSQL schéma public, TenantMiddleware, Sanctum
- Jules : Initialisation Flutter, GoRouter, Riverpod, Dio avec intercepteurs
- Compléter API contrats : /employees/{id}, /departments, /schedules, /sites

□ Semaines 3-4 — Modules core

- Compléter API contrats : /absences, /tasks, /evaluations
- Compléter modèles Dart : Absence, SalaryAdvance, Task
- Documenter stratégie cache Redis dans 01_PROMPT_MASTER_CLAUDE_CODE.md

□ Semaines 5-12 — Modules avancés

- Compléter API contrats : /notifications, /payroll, /reports, Super Admin
- Créer les modèles Dart restants : PayrollSlip, AppNotification, Evaluation
- Créer .github/workflows/deploy.yml
- Créer CHANGELOG.md

CONCLUSION : Votre conception est à 92% — c'est une base remarquable. Les 8% manquants (API contrats + modèles Dart + mocks) sont des tâches de documentation de 2-3 jours, pas de développement. Complétez-les avant de lancer les agents IA.